XORFLOW: Online Inter-Layer Support Compression for Deep Residual GNN Accelerators

HPCA 2027 Submission **#1467**
Confidential Draft
Do NOT Distribute!!

***Abstract*—Deep residual graph neural networks repeatedly materialize sparse intermediate features, making activation movement a major off-chip cost. This paper presents** XORFLOW**, an online lossless representation that exploits positional persistence between adjacent post-ReLU supports.** XORFLOW **independently encodes an anchor support and represents the following support by an XOR event stream only when the complete delta record is smaller than independent encoding. Across ten compatible eight-layer checkpoints,** XORFLOW **reduces serialized support writes by 37.1% and complete physical traffic by 7.9%, yielding 1.083× trace-weighted geometric-mean modeled subsystem speedup. On valid deeper configurations, the modeled subsystem speedup reaches 1.324× on Reddit D16. Byte-exact reconstruction, finite-capacity producer/consumer scheduling, external memory replay, RTL co-simulation, and physical implementation substantiate the representation and datapath.**

Anonymous artifact: https://anonymous.4open.science/r/HPCA-2-0315/.

I. Introduction

Graph neural networks (GNNs) alternate irregular neighborhood aggregation with dense feature transformation. This execution pattern has led to specialized accelerators with separate sparse and dense engines [9], [12], [23]. Deep residual GNNs add another pressure point: every block produces a large intermediate node–feature matrix that must remain available to the next block. As depth grows, these intermediate tensors create a repeated stream of off-chip writes and reads.

ReLU makes many intermediate values zero. A sparse accelerator stores the remaining values together with a *support*, the binary map that identifies their node–feature positions. Prior feature formats, including BEICSR in SGCN [24], reduce this traffic by compressing each support separately. That approach exploits sparsity inside one layer but ignores how the support changes from one residual block to the next.

The key observation behind XORFLOW is that adjacent residual layers often preserve the location of active features. Only a smaller set of positions switches between zero and nonzero. This creates an inter-layer representation opportunity: retain one independently decodable support and encode the next support as the positions that changed. The idea is simple algebraically, but an online accelerator must solve four concrete problems. The first support must be committed before the second exists; the retained support still needs compact storage; an XOR difference can expand on weakly related regions; and reconstruction must complete before sparse aggregation requests the packed values.

XORFLOW solves these problems with a two-stage representation and a matching dataflow. Supports are partitioned into row *tiles* and feature *slices*; a tile/slice is the unit independently selected and decoded. The first support in a pair is the *anchor*. XORFLOW selects the smallest independently decodable anchor among BEICSR, row events, and a majority-prototype codec. The next support is the *target*. Its changed positions form an XOR *delta*; XORFLOW selects the delta only when its complete serialized record is smaller than an independent BEICSR target. Parallel decoders reconstruct a tile-local bitmap that directly gates packed-value reads.

The resulting mechanism is not tied to one host accelerator. It is a support-stream representation and producer/consumer dataflow that can be inserted wherever sparse intermediate features are written and later consumed. The evaluation uses an optimized BEICSR implementation as the direct numerical baseline because it encodes the same support stream under identical caches, memory, precision, and compute resources. Dense bitmap, CSR variants, independent event coding, spatial prototypes, forced deltas, residual/non-residual models, depth, width, and memory-system sweeps isolate the source and limits of the gain.

This paper makes four contributions:

- **Inter-layer support characterization.** Across 26 traced configurations, adjacent supports reach median Jaccard 0.879 and exception density 0.0588; exact-count independent controls fall to 0.327 and rise to 0.4925, respectively.
- **Online anchor–delta representation.** XORFLOW combines a Hamming-optimal majority prototype, exact XOR reconstruction, and local fallback. The anchor decision never reads a future layer, and the serialized pair-write cost is bounded by BEICSR using the same record partition and slice width; the evaluation separately compares independently optimized BEICSR widths.
- **GNN support-stream microarchitecture.** A producer-side cohort engine performs majority construction, XOR event discovery, exact Dense/ID/Gap8 packing, and local format selection. Parallel variable-length decoders, event scatter, and banked support construction expose reconstructed supports to sparse aggregation.
- **Architecture-faithful system evaluation.** Byte-exact records, complete producer and consumer anchor accounting, all ten eligible checkpoints, a shared eight-channel read/write memory model, SCALE-Sim combination service, held-out Ramulator2 validation, pro-

ducer co-simulation, bank-aware decoder modeling, and integrated-cluster routing quantify both benefit and cost.

II. Preliminaries

A. Residual GNN execution

Let $G = (V, E)$ be a graph with $N = |V|$ nodes. Layer ℓ consumes $H^{(\ell)} \in \mathbb{R}^{N \times F_\ell}$ and computes

$$H^{(\ell+1)} = \phi\Big(\widehat{A} H^{(\ell)} W^{(\ell)} + R^{(\ell)}\Big), \tag{1}$$

where $\widehat{A}$ is the normalized sparse adjacency operator, $W^{(\ell)}$ is the dense transform, $R^{(\ell)}$ is the residual branch, and ϕ applies normalization and ReLU [2]. Aggregation traverses graph edges and reads features irregularly; combination applies $W^{(\ell)}$ on dense arrays.

The *support* of a post-ReLU feature matrix is the binary matrix

$$S_\ell[v, f] = \mathbf{1}\Big[H^{(\ell)}[v, f] \neq 0\Big]. \tag{2}$$

The nonzero values are stored densely in a packed stream, while the support tells aggregation which values exist. Existing layer-local formats reduce the size of S_ℓ but encode S_ℓ and $S_{\ell+1}$ independently.

B. Inter-layer support transitions

For adjacent supports, define the transition support

$$D_\ell = S_\ell \oplus S_{\ell+1}. \tag{3}$$

A one in D_ℓ is an *event*: one node–feature position changed between layers. The representation developed below encodes these events when doing so is smaller than independently encoding the target support.

III. XORFLOW Representation

A. Tiles, slices, anchors, and targets

Reverse Cuthill–McKee reorders nodes once for storage locality and is applied identically to every format [8]. Each support is divided into row tiles of $R = 128$ nodes and feature slices of width C. For tile t and slice q, $S_\ell^{t,q} \in \{0,1\}^{R \times C}$ is one independently encoded record.

Non-overlapping layer pairs are indexed by p:

$$A_p = S_{2p}^{t,q}, \qquad T_p = S_{2p+1}^{t,q}, \qquad D_p = A_p \oplus T_p, \tag{4}$$

where A_p is the anchor, T_p is the target, and D_p is their event map. An unpaired final support is encoded independently.

B. Hardware-oriented event coding

Let $E \subseteq \{0, \ldots, U-1\}$ contain k sorted events from a universe of U positions. Let $w(U) = \lceil \log_2 U \rceil$ and $c(U) = \lceil \log_2(U+1) \rceil$. Ignoring record-wide fields common to all three choices, the event payload uses the minimum of

$$L_{\text{dense}}(U) = U, \tag{5}$$

$$L_{\text{id}}(U, k) = c(U) + k\, w(U), \tag{6}$$

$$L_{\text{gap}}(U, k, b) = 9 + b\,[w(U) + 5] + 8(k - b), \tag{7}$$

where b is the number of Gap8 blocks. Dense mode emits a bitmap; ID mode emits fixed-width sorted positions; Gap8 emits one restart identifier and up to 31 positive 8-bit gaps per block. The selector chooses the shortest emitted payload after byte rounding. In particular, ID mode beats a dense bitmap whenever

$$k < \frac{U - c(U)}{w(U)}, \tag{8}$$

which formalizes why sparse transition events are compact even when the anchor itself is not sparse.

C. Independent anchor coding

The anchor must be written before T_p exists. XORFLOW therefore compares three independently decodable records.

BEICSR stores the layer-local bitmap used by the direct baseline. A0 applies Eqs. 5–7 to the set positions of each row. A2 partitions up to $g = 32$ rows into a cohort, stores one prototype $P \in \{0,1\}^C$, and event-encodes each residual row $A_p[r, :] \oplus P$.

The prototype minimizes residual Hamming count. For cohort rows $x_1, \ldots, x_g$, define the residual event count

$$E(P) = \sum_{r=1}^{g} \|x_r \oplus P\|_0. \tag{9}$$

Proposition 1 (Hamming-optimal majority prototype)**.** *Let* $n_j = \sum_{r=1}^{g} x_r[j]$. *The bitwise majority*

$$P^*[j] = \mathbf{1}[n_j > g/2] \tag{10}$$

minimizes $E(P)$*, with deterministic zero ties, and*

$$E(P^*) = \sum_{j=1}^{C} \min(n_j, g - n_j) \leq \frac{gC}{2}. \tag{11}$$

Proof. Equation 9 separates by feature column. Choosing $P[j] = 0$ creates n_j residual events; choosing $P[j] = 1$ creates $g - n_j$. The smaller choice is the majority value, yielding $\min(n_j, g - n_j)$ events for column j. Summing over columns proves optimality. Since each minimum is at most $g/2$, the total is at most $gC/2$. □

The proposition optimizes the number of residual events, not the nonlinear serialized-byte objective after Dense/ID/Gap8 selection and byte rounding. XORFLOW therefore calls the prototype Hamming-optimal rather than byte-optimal. The majority construction is a one-pass, column-separable hardware rule with the event-count bound in Eq. 11; complete serialized lengths are still compared after every candidate is emitted.

Let $B_F(X)$ denote the complete 64-byte-aligned serialized length of support X under format F, including its record header and variable-length fields. The anchor selector is

$$F_A = \operatorname*{arg\,min}_{F \in \{\text{BEICSR}, \text{A0}, \text{A2}\}} B_F(A_p), \tag{12}$$

with ties resolved in favor of BEICSR. The selected anchor is committed immediately.

D. Target delta and local fallback

When T_p is produced, the encoder obtains A_p from the anchor buffer or a charged memory reread and forms D_p. The target choice is

$$F_T = \begin{cases} \text{DELTA}, & B_{\text{DELTA}}(D_p) < B_{\text{BEICSR}}(T_p), \\ \text{BEICSR}, & \text{otherwise.} \end{cases} \tag{13}$$

The delta is therefore used only where its complete serialized record is beneficial.

Proposition 2 (Fixed-partition serialized pair-write bound)**.** *Fix the same tile boundaries, slice width, record headers, and alignment policy for* XORFLOW *and* BEICSR*. The serialized support writes for every pair satisfy*

$$B_{\text{XF}}(A_p, T_p) \le B^{(q)}_{\text{BEI}}(A_p) + B^{(q)}_{\text{BEI}}(T_p), \tag{14}$$

where q denotes that common partition and slice width.

Proof. Under the fixed layout q, the candidate set in Eq. 12 contains BEICSR, so the selected anchor is no larger than $B^{(q)}_{\text{BEICSR}}(A_p)$. The target candidate set in Eq. 13 also contains BEICSR, so the selected target is no larger than $B^{(q)}_{\text{BEICSR}}(T_p)$. Adding the two inequalities proves Eq. 14. □

The bound applies only to serialized support writes under one fixed layout. It neither compares against the independently optimized per-workload BEICSR width nor bounds reads, packed values, topology, output writes, or cache effects. Those quantities are evaluated separately.

E. Exactness, causality, and bounded state

A delta target reconstructs as

$$A_p \oplus D_p = A_p \oplus (A_p \oplus T_p) = T_p. \tag{15}$$

The anchor decision is a function of A_p alone and completes before T_p is available. The target decision reads only the committed anchor and current target. The producer and consumer have distinct anchor lifetimes. The producer retains or reconstructs A_p until T_p has been encoded; the consumer must later obtain A_p again before decoding a DELTA target. No record refers beyond its pair, so the dependency does not form a multi-layer chain, but the consumer-side anchor source must be explicit and timing-active.

F. Serialized record

Each record begins with a 16-bit little-endian header containing a 2-bit format tag and a 14-bit payload-byte count. Payload integers are unsigned and emitted least-significant-bit first. An event payload begins with a 2-bit mode. Dense mode emits U bits; ID mode emits a $c(U)$-bit count and $w(U)$-bit identifiers; Gap8 emits a 9-bit block count, then for each block a $w(U)$-bit restart identifier, a 5-bit length-minus-one, and positive 8-bit gaps. A2 emits one C-bit prototype per cohort followed by residual event payloads. A 16-byte offset table locates variable-length records, and every record is zero-padded to 64 bytes. The decoder rejects reserved tags, truncated fields, zero gaps, unsorted or out-of-range identifiers, inconsistent counts, dimension mismatches, and nonzero padding.

The serialized format is self-delimiting and byte-exact: analytical field lengths match emitted records, and decoding reconstructs each support without reference to layers outside its anchor–target pair.

Algorithm 1 Online encoding of one tile/slice stream

Require: Supports $S_0, S_1, \ldots$ in production order
for $\ell = 0, 1, \ldots$ **do**
  if ℓ is even **then**
    $A \leftarrow S_\ell$
    build BEICSR(A) and A0(A)
    for each 32-row cohort, compute P^* by Eq. 10
    build A2(A) from P^* and row residual events
    commit the shortest of BEICSR(A), A0(A), and A2(A)
    retain A subject to anchor-buffer capacity
  else
    obtain anchor A; charge a memory read on a miss
    $D \leftarrow A \oplus S_\ell$
    build DELTA(D) using Eqs. 5–7
    build independent BEICSR(S_ℓ)
    commit the shorter target record; prefer BEICSR on a tie
    release A
  end if
end for

IV. Microarchitecture and Performance Model

A. Producer-side encoder

The write path in Figure 2 receives the post-ReLU support tile while packed values follow the ordinary output path. For an anchor, parallel generators construct BEICSR, A0, and A2 records and their exact byte counts. For a target, independent BEICSR generation runs beside anchor XOR, event discovery, and DELTA packing. A byte selector commits the minimum record, and the output FIFO issues 64-byte-aligned memory writes.

The bounded cohort kernel captures a 32×64-bit support region, accumulates the majority prototype, scans all 2,048 positions, and constructs Dense, ID, and Gap8 streams. Twenty-four reference records spanning empty, dense, majority-tie, Gap8-boundary, diagonal, and randomized inputs match the software serializer byte-for-byte under randomized stalls. The selector/ready–valid boundary maps to 810 Yosys cells, and the segmented tile-record engine maps to 1,055 cells. This mapped unit performs event discovery, exact candidate-length selection, descriptor offsets, and finite ready/valid output for one 2,048-bit tile record. Producer service and backpressure are included in the capacity-constrained execution model.

B. Producer and consumer anchor lifecycles

Every DELTA target uses the uncompressed anchor bitmap twice. The producer needs A_p before forming $D_p = A_p \oplus T_p$;

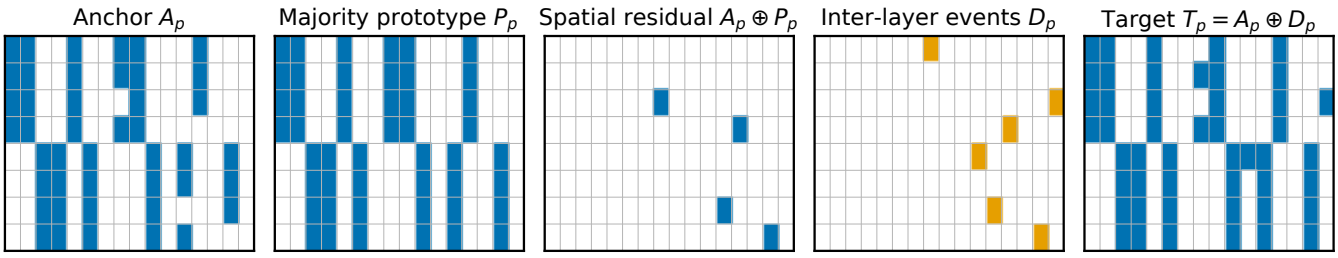

Fig. 1. Bit-exact example. The majority prototype minimizes the four anchor residual events. Six inter-layer events form D_p, and XOR with the committed anchor reconstructs T_p exactly.

Figure 2 placeholder

Final producer–consumer microarchitecture artwork will be inserted here.

Fig. 2. Placeholder for the final XORFLOW producer and consumer microarchitecture.

the consumer later needs the same A_p before reconstructing $T_p = A_p \oplus D_p$. The two lifecycles have separate buffers, reads, decode service, and readiness conditions.

A default 128×128 tile/slice contains $RC = 16{,}384$ support bits, or 2 KiB. Both evaluated stores use record identifiers and least-recently-used replacement. A producer entry is live from anchor production until target encoding. A consumer entry is inserted after anchor decode and remains live until the paired target completes or eviction. Let h_p^P and h_p^C denote producer and consumer hits. A miss rereads the exact padded committed anchor record and decodes it:

$$R_p^P = (1 - h_p^P)\, 64 \lceil B_{F_A}(A_p)/64 \rceil , \tag{16}$$

$$R_p^C = (1 - h_p^C)\, 64 \lceil B_{F_A}(A_p)/64 \rceil , \tag{17}$$

$$C_p^x = (1 - h_p^x) \left[C_{\mathrm{mem}}(R_p^x) + C_{\mathrm{adec}}(F_A, A_p) \right] . \tag{18}$$

Here $x \in \{P, C\}$. An independently encoded target requires neither consumer anchor read nor reconstruction XOR.

Across all evaluated traces, the lifecycle accounting classifies 72,604 DELTA targets: 422 find a decoded consumer anchor and 72,182 reread the committed record. At 16 KiB, the consumer therefore has a 0.581% hit rate, rereads 149.60 MB (142.67 MiB), and performs 1.320 million anchor-decode cycles. Producer accounting reconciles to the same 149.60 MB and 1.320 million cycles; together the two lifecycles charge 299.20 MB and 2.641 million decode cycles. Increasing the consumer store to 256 KiB, 1 MiB, and 4 MiB raises hit rate to 7.10%, 27.94%, and 80.43%, while reread traffic falls to 133.46, 103.92, and 28.61 MiB. The store is bounded pair state rather than a conventional high-reuse cache: each anchor serves one paired target, and large graphs stream many records between production and consumption.

The final unified event trace contains 73,652 target records and verifies the complete lifecycle ordering. On every producer miss, the anchor reread completes and the anchor decode finishes before target XOR and serialization begin; the minimum memory-to-decode and decode-to-encode slack is nonnegative, with zero violations. On the consumer path, target reconstruction waits for both the complete target payload and the decoded anchor, and sparse aggregation waits for support-cache completion. Producer and consumer reread bytes and decode cycles reconcile exactly with the per-record trace.

C. Parallel reconstruction

The read path contains four eight-lane clusters. Each 64-bit lane parses one record, expands event positions, applies a prototype or anchor XOR, and emits support-word updates. The stream router distributes variable-length records; event-scatter logic maps updates to cache words; same-word updates are merged before finite bank arbitration. A 16-KiB banked cache holds reconstructed tile supports. A row prefix is the cumulative number of set support bits before a row; it converts a support position into its offset in the packed-value stream. Aggregation requests packed values only after the corresponding support row and prefix are ready.

The cycle model includes finite lane FIFOs, stream imbalance, event scatter, bank conflicts, same-word merging, SRAM-port constraints, and downstream ready/valid backpressure. In the bank sweep, median achieved throughput rises from 949.6 encoded bits/cycle with eight banks to 1,559.1 with

16 banks and 1,822.4 with 32 banks; conflicts fall by 97.1% relative to eight banks and lane utilization reaches 89.0%. An integrated eight-lane decoder/support-cache top passes Verilator co-simulation and routes in OpenROAD/Nangate45 at a 1.0-ns target with +0.565-ns slack, zero detailed-route DRC errors, and a 0.0682 mm^2 die. Supplementary Fig. S4 reports the encoder queue and decoder-bank surfaces.

D. Capacity-enforcing execution schedule

For record i at stage j, let $r_j(i)$ be data readiness, $w_j(i)$ the earliest resource-availability time, and $q_j(i)$ the time at which finite downstream capacity becomes available. The schedule uses

$$s_j(i) = \max\{r_j(i), w_j(i), q_j(i)\}, \tag{19}$$

$$f_j(i) = s_j(i) + L_j(i). \tag{20}$$

Target encoding waits for both the produced target support and the producer anchor,

$$r_E(i) = \max\{f_{\text{support}}(i), f^P_{\text{anchor}}(i)\}, \tag{21}$$

and a DELTA target waits for the complete target record and decoded consumer anchor,

$$r_D(i) = \max\{f_{\text{desc}}(i), f_{\text{payload}}(i), f^C_{\text{anchor}}(i)\}. \tag{22}$$

Input, producer, decode, aggregation, combination, and write-back queues have depth four and block their producers when full. Layer $\ell + 1$ cannot consume data until every write from layer ℓ completes.

One persistent memory resource spans the full run and is shared by producer-anchor reads, target and consumer-anchor reads, and output writebacks. A 32-entry request queue is distinct from eight timing-active channel slots. Each 64-byte record is deterministically striped by physical address; a channel transfers 32 bytes/cycle, applies a 50-cycle callback latency, and charges eight cycles when direction changes between reads and writes. This model intentionally abstracts bank and row scheduling. A calibration trace sets the timing scale, and an independent held-out trace differs from Ramulator2 by 0.45% (Section VI-H). Thus the event graph captures shared read/write contention and finite admission, while the external simulator checks the banked HBM timing abstraction.

Combination service is not inferred from row count. Every executed record looks up a SCALE-Sim [20] entry for a 32×32 weight-stationary array using its (M, K, N) feature shape. Seven unique shapes cover the evaluated records; the cache records dimensions, cycles, prefetch cycles, utilization, and a SHA-256 key. Baseline and XORFLOW consume the same per-shape table. An independent max-plus implementation reproduces all baseline and proposal schedules across the twelve reported configurations.

E. From compressed bytes to speedup

Let the baseline physical traffic be $T_B = T_o + T_s$, where T_s is support traffic and T_o is unchanged topology, value, descriptor, and output traffic. If support encoding reduces T_s by fraction ρ and anchor rereads add T_r, then

$$\tau = 1 - \frac{T_X}{T_B} = \alpha\rho - \delta, \quad \alpha = \frac{T_s}{T_B}, \quad \delta = \frac{T_r}{T_B}. \tag{23}$$

Thus a large support reduction produces a large total reduction only when support occupies a meaningful fraction of physical traffic and rereads remain small.

Let $C_B = C_{\text{fixed}} + C_{\text{mem}}$ be baseline subsystem cycles. Let $\mu = C_{\text{mem}}/C_B$ be the fraction exposed to traffic reduction, and let $\epsilon = C_{\text{codec}}/C_B$ be encoding, decoding, and added stall cost. A serial conservative model gives

$$\text{Speedup} = \frac{C_B}{C_{\text{fixed}} + (1-\tau)C_{\text{mem}} + C_{\text{codec}}} = \frac{1}{1 - \mu\tau + \epsilon}. \tag{24}$$

The necessary and sufficient break-even condition in this model is

$$\mu\tau > \epsilon. \tag{25}$$

This relation predicts the evaluation trends: depth raises the amount of reusable support traffic, narrow slices suffer fixed record overhead, and very wide layers can reduce bytes while dense combination lowers μ and hides the latency benefit.

The cycle evaluation reports the aggregation–combination subsystem under Eq. 20. Baseline and XORFLOW use the same queue capacities, barriers, and compute resources. The reported timing result is

$$\text{Speedup} = \frac{C_B}{C_X}, \tag{26}$$

where C_B and C_X are the corrected baseline and XORFLOW completion times from the same finite-queue event graph.

V. Evaluation Methodology

A. Workloads

The principal architecture is an eight-layer, width-128 residual GCN with LayerNorm–ReLU–GCN ordering. The primary evaluation contains ten checkpoints spanning Flickr, OGBN-Arxiv, Reddit, and Yelp, with three independent runs for each of the first three datasets and one for Yelp. Additional traces vary depth, width, residual structure, GNN operator, and graph. Supports are captured after FP8 E4M3 quantization in layer-production order, and paired FP32 versus FP8/FP16 quality uses the same checkpoint and run.

All ten primary checkpoints use the same optimized baseline, serializer, producer/consumer anchor policy, finite queues, shared eight-channel memory resource, SCALE-Sim table, and hardware service rates. A 16-layer Arxiv model and Chameleon negative control use the same scheduler but are reported separately from the primary aggregate.

B. Baselines and common host

All comparisons use the same traces, node order, FP8/FP16 arithmetic, cache hierarchy, memory transactions, topology stream, and compute resources. The primary baseline independently selects the best BEICSR slice width from 64, 96, 128, and 256 features per workload. Dense bitmap, CSR

TABLE I
PRIMARY DATASETS AND INPUT DIMENSIONS.

Dataset	Nodes	Edges	Input features	Classes
Reddit	232,965	114,615,892	602	41
OGBN-Arxiv	169,343	1,166,243	128	40
Flickr	89,250	899,756	500	7
Yelp	716,847	13,954,819	300	100

variants, independent row events, independent majority prototypes, fixed-anchor XOR, generic XOR–gap coding, forced delta, and full XORFLOW provide matched representation comparisons. A noncausal pair oracle is used only as an upper-bound diagnostic.

The modeled host runs at 1 GHz with eight aggregation engines, eight dense combination arrays, a 512-KiB feature cache, and a 16-KiB support cache. Eight HBM2 channels provide a nominal 256 GB/s with 32-byte transactions. The default support record is 128×128 bits, and the logical decoder contains four clusters of eight 64-bit lanes.

C. Metrics and validation

Support bytes are exact emitted records. Physical traffic includes topology, packed values, support records, descriptors, output writes, and compressed-anchor reads on both the producer and consumer paths. The unified accounting charges both producer and consumer compressed-anchor reads. Support and lifecycle bytes are exact emitted-record quantities; subsystem cycles come from the common finite-queue event graph described above. The result package contains per-trace absolute cycles, dependency and queue audits, the SCALE-Sim shape cache, RTL co-simulation, synthesis, routing, and held-out external-memory validation. Ramulator2 [15] provides the absolute HBM completion check, while DRAMsim3 [13] independently verifies complete request service under a second mapping. Yosys, Verilator, OpenROAD [1], and CACTI [16] provide implementation evidence.

VI. EVALUATION

A. Inter-layer support persistence

Figure 3(a–b) isolates positional persistence from sparsity. Across the ten primary checkpoints, real adjacent pairs reach median support Jaccard 0.933 and exception density 0.0356. Exact-count independent masks preserve each support's number of ones but reduce median Jaccard to 0.344 and raise exception density to 0.496. Row permutation raises exception density to 0.370 and causes independent encoding to be selected for essentially every record. Across the broader 26-trace suite, the corresponding medians are 0.879 versus 0.327 and 0.0588 versus 0.4925. The delta advantage therefore derives from learned positional correspondence rather than density alone; Supplementary Fig. S1 reports the complete distributions and layerwise progression.

B. Anchor accounting and dependency audit

Figure 4(a) shows that the 16-KiB consumer store retains only 0.581% of delta anchors. Every miss rereads a compressed committed record rather than an uncompressed 2-KiB bitmap. Increasing capacity to 4 MiB raises the hit rate to 80.43% and lowers consumer rereads from 142.67 to 28.61 MiB. The producer and consumer totals reconcile exactly, and no delta target is unclassified. Figure 4(b) broadens external timing evidence to Arxiv, Flickr, Reddit, and Yelp retained streams while distinguishing the independently held-out Flickr point.

The per-record audit covers 73,652 targets and reports zero producer-readiness or consumer-classification failures. On every producer miss, compressed-anchor memory completion and decode precede target XOR and serialization. Every one of the 72,604 delta targets has an explicit consumer source. The common event model covers all ten eligible checkpoints, and its independent recurrence reproduces every baseline and proposal completion exactly.

C. Primary representation result

Across all ten compatible eight-layer checkpoints, XORFLOW reduces serialized support writes by 37.05% and complete physical traffic by 7.94% on average. Dataset support reductions are $56.74 \pm 1.40\%$ on Flickr, $19.91 \pm 1.49\%$ on Arxiv, $31.77 \pm 4.03\%$ on Reddit, and 45.24% on Yelp. The final common scheduler achieves $1.083\times$ trace-weighted and $1.076\times$ dataset-balanced geometric-mean modeled subsystem speedup, with a maximum of $1.279\times$. Dataset geometric means are $1.008\times$ on Flickr, $1.103\times$ on Arxiv, $1.154\times$ on Reddit, and $1.046\times$ on Yelp.

Eight of ten checkpoints improve. Across independent runs, Flickr ranges from $0.975\times$ to $1.055\times$, showing that strong support compression may remain near break-even when unchanged feature and output traffic dominate. Reddit reaches a maximum of $1.279\times$, and the 16-layer Arxiv extension reaches $1.220\times$ versus $1.097\times$ for the corresponding eight-layer configuration. The Chameleon control remains near break-even at $0.994\times$, consistent with its weaker residual correspondence. Complete per-checkpoint bytes and absolute cycles appear in Supplementary Table S1.

Table II separates logical support compression from its system-level effect. The largest gains occur on memory-exposed Reddit, whereas Flickr shows that even strong support compression cannot overcome dominant unchanged traffic.

D. Codec decomposition

The common serializer and consumer-byte accounting separate independent spatial coding from temporal coding. Relative to optimized BEICSR, independent A2 reduces aggregate physical bytes by 5.4%; fixed-anchor XOR reaches 10.5%; generic XOR–gap coding reaches 10.4%; forced delta reaches 19.6%; and complete online XORFLOW reaches 12.7%. Temporal coding is the largest byte reduction beyond independent spatial coding. Forced delta is smaller in aggregate but loses local protection and the fixed-partition pair-write bound;

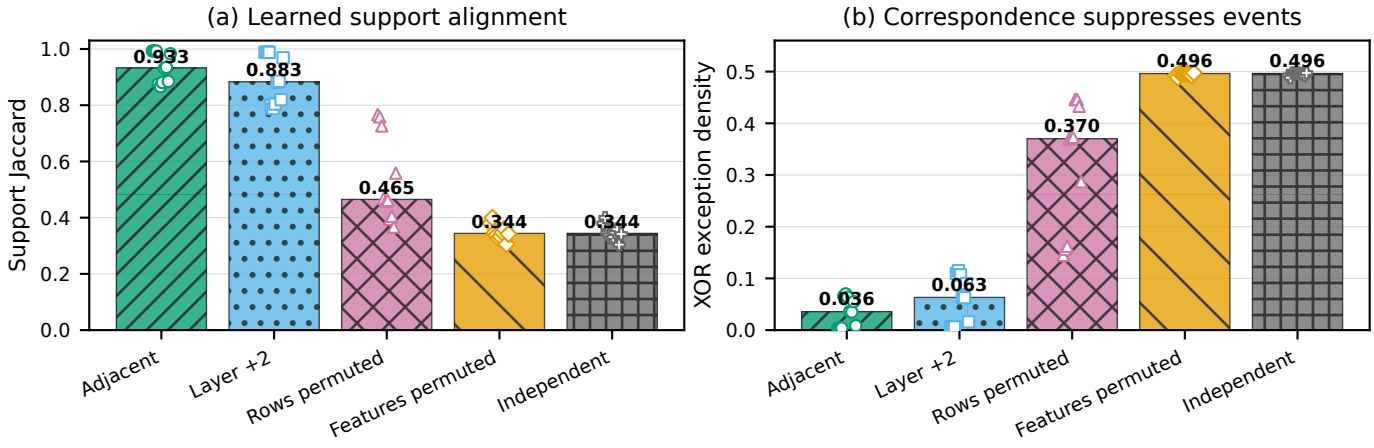

Fig. 3. Inter-layer support persistence. (a) Real adjacent supports preserve substantially more active positions than nonadjacent, row-permuted, feature-permuted, and exact-count independent controls. (b) Destroying correspondence raises XOR exception density. Bars show medians across the ten compatible eight-layer checkpoints; markers show individual checkpoints. Full 26-trace distributions and layerwise results appear in Supplementary Fig. S1.

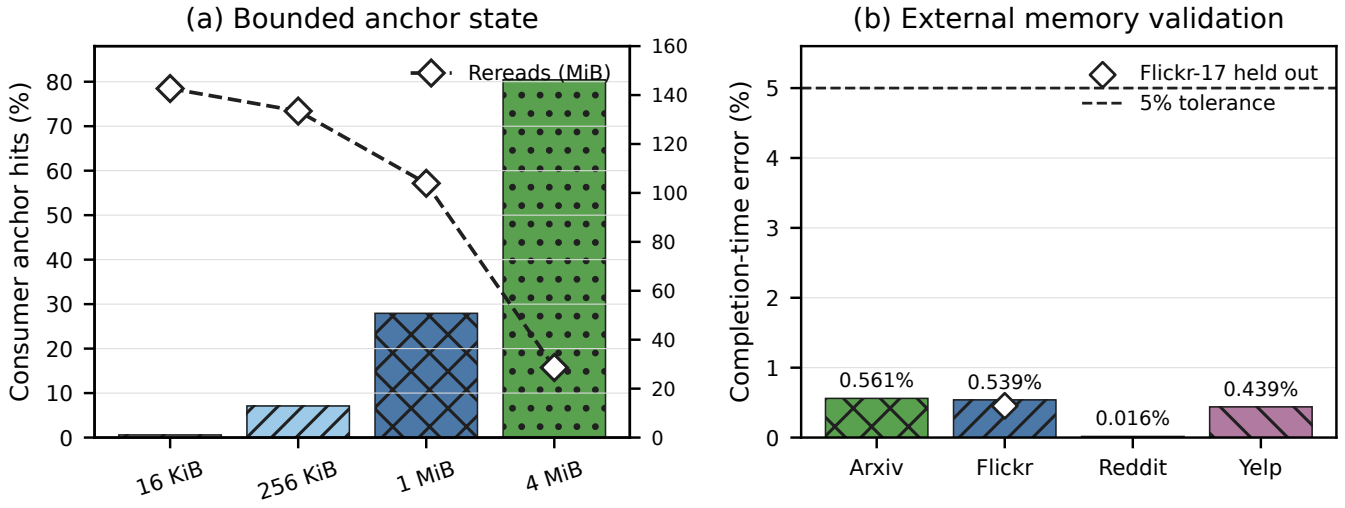

Fig. 4. Online and timing validation. (a) Larger anchor stores reduce exact compressed rereads, although each anchor has only one paired target. (b) Cross-workload complete retained-stream replay remains within 0.57% of Ramulator2; the overlaid Flickr–17 marker is an independent held-out absolute check at 0.45%. All are below the predeclared 5% tolerance.

complete XORFLOW pays for independently decodable anchors and local fallback.

The ablation compares serialized and physical bytes under common serializer, padding, unchanged-traffic, and anchor-accounting rules. Supplementary Fig. S2 reports the complete byte decomposition.

E. Operating regime

The operating regime follows Eq. 25. Residual connections preserve support correspondence, while exact-count independent, row-permuted, feature-permuted, and non-residual controls sharply increase exception density. The 16-layer Arxiv result shows that additional residual pairs can expose more removable support traffic, whereas the Chameleon control and two weak Flickr checkpoints show that compression alone cannot overcome a small exposed-memory fraction. Supplementary Figs. S1 and S5 and Supplementary Secs. VIII–IX report the layerwise, slice, cache, residual/non-residual, and quality evidence.

Figure 5 connects the layerwise persistence in Supplementary Fig. S1 to subsystem behavior: valid Reddit improves monotonically from 1.279× at D8 to 1.324× at D16, while Arxiv exceeds 1.2× at D16. Flickr's 0.981× D16 result bounds the claim—depth helps only when positional persistence coincides with exposed memory traffic.

F. Exactness and numerical quality

Every evaluated stream reconstructs the original support bit-for-bit, so XORFLOW adds no numerical error. The paired quality difference comes only from the FP8/FP16 arithmetic

TABLE II

FINAL PRIMARY RESULTS ACROSS ALL TEN COMPATIBLE EIGHT-LAYER CHECKPOINTS. REDUCTIONS ARE ARITHMETIC MEANS; SPEEDUP IS THE GEOMETRIC MEAN WITHIN EACH DATASET. THE RANGE RETAINS EVERY SEED, INCLUDING REGRESSIONS.

Dataset	Checkpoints	Support reduction	Physical-traffic reduction	Subsystem speedup	Per-seed range
Flickr	3	56.74%	1.39%	1.008×	0.975–1.055×
Arxiv	3	19.91%	10.37%	1.103×	1.095–1.117×
Reddit	3	31.77%	13.10%	**1.154**×	1.082–**1.279**×
Yelp	1	45.24%	4.80%	1.046×	1.046×
Overall	10	37.05%	7.94%	1.083×	0.975–**1.279**×

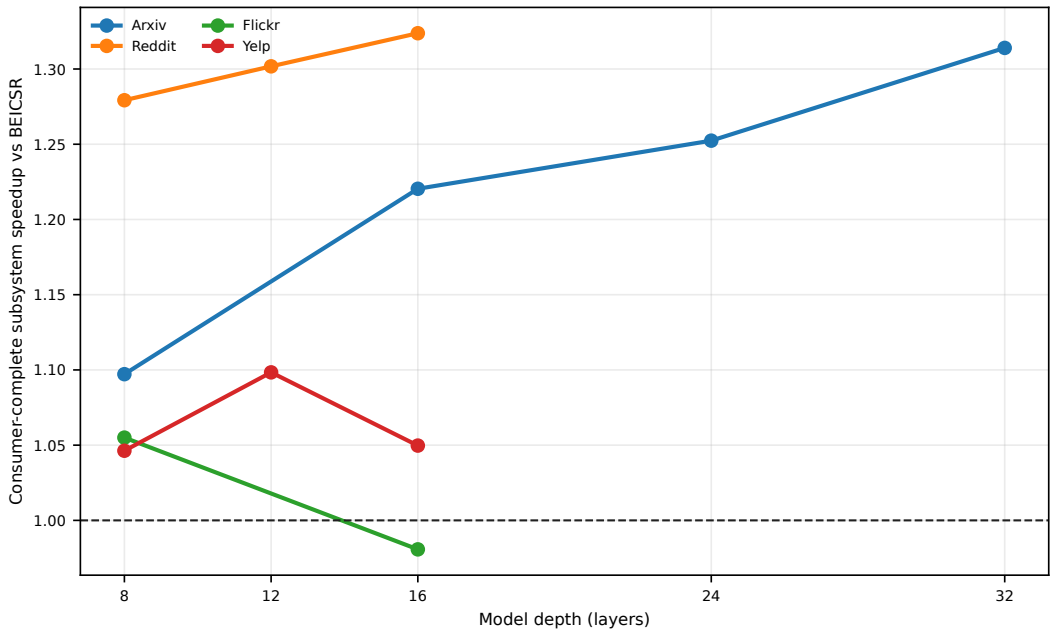

Fig. 5. Depthwise consumer-complete sensitivity. Every point is an independently trained model; the D12+ extensions charge producer and consumer anchor recovery and pass exact recurrence checks. Arxiv D24/D32 are borderline-quality diagnostics, Yelp D16 is invalid, and Flickr D16 is a valid negative control; these points do not enter primary aggregates.

shared with the baseline. Across compatible primary checkpoints, FP8/FP16 minus FP32 accuracy ranges from -0.053 to $+0.031$ percentage points. Supplementary Table S6 reports every compatible primary pair; Supplementary Sec. IX reports the deeper and negative-control configurations.

G. RTL and physical implementation

The selector boundary maps to 810 Yosys cells, and the bounded tile front end maps to 1,055 cells. That front end captures a 32-row by 64-feature tile, accumulates the majority mask, discovers XOR events, and exposes a finite ready/valid interface. A separate 2,048-bit stream RTL constructs Dense, fixed-ID, and Gap8 payloads with descriptor offsets; the serialized ready/valid interface reproduces 24 reference streams under randomized stalls. On the consumer side, 32 banks sustain 1,822.4 encoded bits/cycle and reduce conflicts by 97.1% relative to eight banks. The integrated eight-lane top routes at a 1.0-ns target with +0.565-ns slack, zero detailed-route DRC errors, 13.881 mm of wire, a 0.0682 mm^2 die, and 0.001795 mm^2 of standard cells. The 16-KiB CACTI support SRAM occupies 0.0549 mm^2 with 0.374-ns access and 0.00891 nJ/read.

H. External memory replay

The internal completion model uses one shared eight-channel resource, but abstracts bank state and row scheduling. Ramulator2 therefore supplies an independent absolute check with HBM2, cache-line interleaving, RoBaRaCoCh mapping, FR-FCFS scheduling, open rows, refresh, and finite read/write queues. The timing scale is calibrated on one complete request stream and evaluated on an independent held-out stream. The model predicts 4,064,919 cycles, while Ramulator2 reports 4,083,428 cycles, an absolute error of 0.453%. Both streams contain producer reads, consumer-anchor reads, and writebacks, directly testing the shared-memory concurrency abstraction.

DRAMsim3 independently accepts and drains representative real and adversarial streams under a second address mapping. DRAMsim3 confirms complete request service under its independent address mapping. Supplementary Table S5 reports the

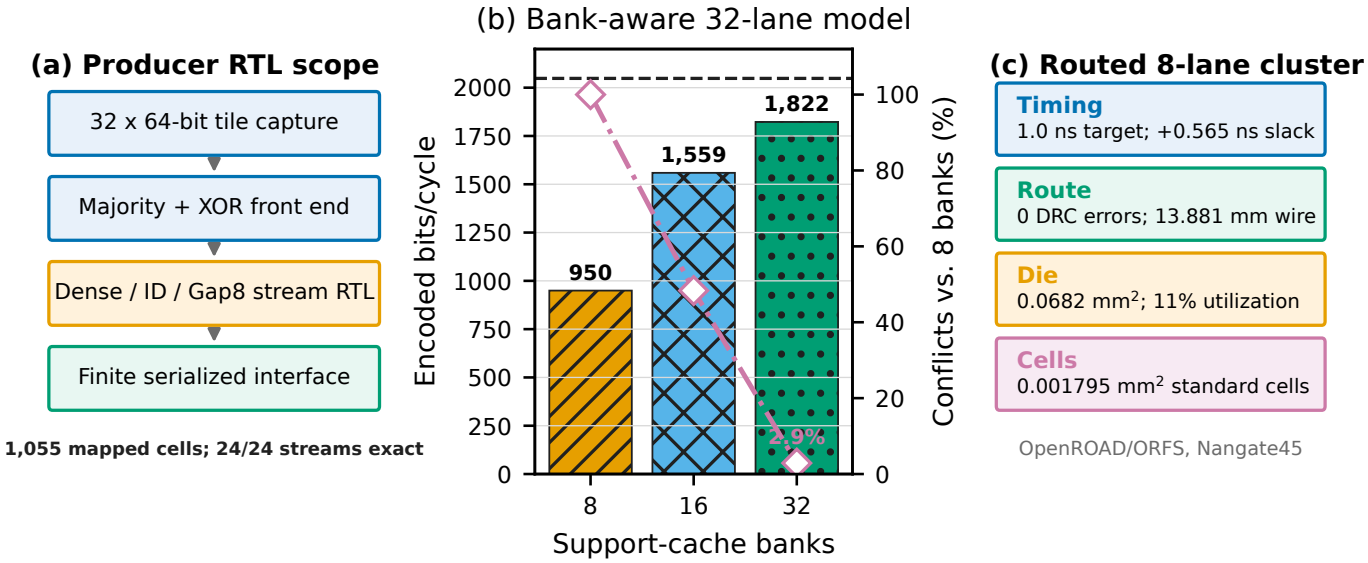

Fig. 6. Implementation evidence. (a) The mapped tile front end performs majority construction and XOR-event discovery; a separate finite stream RTL packs Dense/ID/Gap8 records, while the serialized ready/valid interface matches 24 reference streams. (b) Increasing support-cache banks raises modeled 32-lane throughput from 950 to 1,822 encoded bits/cycle. (c) An integrated eight-lane decoder/support-cache top routes cleanly in Nangate45.

held-out comparison, request counts, and the seven SCALE-Sim shapes used by the execution model.

VII. Related Work

Deep residual GNNs and accelerator dataflows. Residual and initial-residual formulations such as DeepGCNs and GCNII make substantially deeper GNNs trainable [5], [11]. Most GNN accelerators instead target topology irregularity, aggregation–combination imbalance, or dataflow. HyGCN separates sparse aggregation from dense combination [23]; GCNAX searches loop order, tiling, and fusion [12]; ReGNN eliminates redundant message-passing work [4]; GROW specializes sparse–dense multiplication [9]; and BeaconGNN reorders storage-side streams [22]. These mechanisms determine when and where features move, but they do not define a differential representation for positionally aligned supports from successive residual blocks. XORFLOW is complementary: it changes the support stream presented to the host dataflow rather than the graph traversal or dense-compute schedule.

Sparse and compressed GNN activations. SGCN is the closest accelerator precedent. It observes high post-ReLU sparsity in deep residual GCNs, introduces BEICSR to encode each feature stream in place, and adds a sparse aggregation path that consumes the compressed representation [24]. XORFLOW retains that execution contract and uses an optimized BEICSR implementation as its direct baseline, but adds a second redundancy axis: correspondence of support positions across adjacent layers. GNN training systems pursue a different objective. EXACT quantizes activations saved for backpropagation to reduce memory, accepting an accuracy and runtime trade-off, while CompressGNN applies hierarchical compression to reduce training redundancy [6], [14]. Those methods compress activation state retained for optimization; XORFLOW losslessly encodes only the Boolean inference support, leaves packed nonzero values unchanged, and reconstructs the support before irregular aggregation.

Activation compression in DNN accelerators. SCNN maintains sparse weights and ReLU activations in compressed form, while cDMA compresses sparse activations offloaded during DNN training [17], [19]. CompAct uses schedule-aware run-length coding, including sparse and lossy variants, for on-chip CNN activation buffers [26]. Lossless codecs such as EBPC and Lane Compression exploit bit-plane correlation or small value deltas within one tensor [3], [10]. Transform-based schemes such as JPEG-ACT and the accelerator of Shao et al. compress a feature map transferred between layers, but use lossy spatial transforms and quantization rather than differential coding between two layer outputs [7], [21]. Thus, "interlayer feature-map compression" in that literature identifies where the compressed tensor sits in the network dataflow; it does not by itself exploit persistence across layer-indexed supports. XORFLOW instead operates on aligned Boolean supports and preserves bit-exact inference semantics.

Differential and temporal representations. Base/delta and XOR constructions are established memory-compression primitives. BDI represents nearby words relative to a base, whereas bit-plane codecs encode local deltas before packing [3], [18]. Dynamic-GNN accelerators also exploit change, but along the graph-snapshot axis: Cambricon-DG removes redundant work between evolving graph states [25]. XORFLOW applies differential coding along the layer axis of one static-graph inference. Its novelty is not XOR itself, but the causal anchor–target protocol, independently decodable anchor selection, complete-record fallback, fixed-partition non-expansion bound, and explicit producer/consumer anchor lifecycle required to expose reconstructed supports to sparse

aggregation.

VIII. Conclusion

XORFLOW exploits measured inter-layer support persistence with a causal anchor–delta representation, exact reconstruction, a Hamming-optimal spatial prototype, and complete-record fallback. Across every compatible eight-layer checkpoint, it reduces serialized support writes by 37.1% and complete physical traffic by 7.9%. A common finite-queue schedule with explicit producer and consumer anchor recovery, one shared eight-channel read/write memory resource, and SCALE-Sim combination service reaches 1.083× trace-weighted and 1.076× dataset-balanced geometric-mean modeled aggregation–combination-subsystem speedup, with a maximum of 1.279×. A held-out Ramulator2 replay validates absolute memory completion within 0.45%. Producer stream equivalence, bank-aware reconstruction, and a routed eight-lane cluster establish a concrete implementation path. These results show that inter-layer support persistence is a measurable architectural property whose benefit survives causal online encoding, anchor recovery, physical-transfer accounting, shared memory contention, and dense-combination timing.

References

[1] T. Ajayi, D. Blaauw, T.-B. Chan, C.-K. Cheng, V. A. Chhabria, D. K. Choo, M. Coltella, S. Dobre, R. G. Dreslinski, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim, J. Li, Z. Liang, U. Mallappa, P. Penzes, G. Pradipta, S. Reda, A. Rovinski, K. Samadi, S. S. Sapatnekar, L. Saul, C. Sechen, V. Srinivas, W. Swartz, D. Sylvester, D. Urquhart, L. Wang, M. Woo, and B. Xu, "OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain," in *Government Microcircuit Applications and Critical Technology Conference (GOMACTech)*, 2019, pp. 1105–1110. [Online]. Available: https://par.nsf.gov/biblio/10171024-openroad-toward-self-driving-open-source-digital-layout-implementation-tool-chain

[2] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016. [Online]. Available: https://arxiv.org/abs/1607.06450

[3] L. Cavigelli, G. Rutishauser, and L. Benini, "EBPC: Extended bit-plane compression for deep neural network inference and training accelerators," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 4, pp. 723–734, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8886603

[4] C. Chen, K. Li, Y. Li, and X. Zou, "ReGNN: A redundancy-eliminated graph neural networks accelerator," in *IEEE International Symposium on High-Performance Computer Architecture*, 2022, pp. 429–443. [Online]. Available: https://ieeexplore.ieee.org/document/9773273

[5] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and deep graph convolutional networks," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 1725–1735. [Online]. Available: https://proceedings.mlr.press/v119/chen20v.html

[6] Z. Chen, F. Zhang, Y. Xia, W. Zhang, X. Zhu, W. Chen, and X. Du, "CompressGNN: Accelerating graph neural network training via hierarchical compression," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025, pp. 286–297. [Online]. Available: https://dl.acm.org/doi/10.1145/3711896.3736888

[7] R. D. Evans, L. Liu, and T. M. Aamodt, "JPEG-ACT: Accelerating deep learning via transform-based lossy compression," in *ACM/IEEE International Symposium on Computer Architecture*, 2020, pp. 860–873. [Online]. Available: https://ieeexplore.ieee.org/document/9138963

[8] J. A. George, "Computer implementation of the finite element method," Computer Science Department, Stanford University, Tech. Rep. STAN-CS-71-208, February 1971. [Online]. Available: https://bitsavers.org/pdf/stanford/Stanford_CS_TR_Collection_2025-12-12/PDF/1971/CS-TR-71-208.pdf

[9] R. Hwang, M. Kang, J. Lee, D. Kam, Y. Lee, and M. Rhu, "GROW: A row-stationary sparse-dense GEMM accelerator for memory-efficient graph convolutional neural networks," in *IEEE International Symposium on High-Performance Computer Architecture*, 2023, pp. 42–55. [Online]. Available: https://ieeexplore.ieee.org/document/10070983

[10] Y. Ko, A. Chadwick, D. Bates, and R. Mullins, "Lane compression: A lightweight lossless compression method for machine learning on embedded systems," *ACM Transactions on Embedded Computing Systems*, vol. 20, no. 2, pp. 16:1–16:26, 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3431815

[11] G. Li, M. Müller, A. Thabet, and B. Ghanem, "DeepGCNs: Can GCNs go as deep as CNNs?" in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9267–9276. [Online]. Available: https://openaccess.thecvf.com/content_ICCV_2019/html/Li_DeepGCNs_Can_GCNs_Go_As_Deep_As_CNNs_ICCV_2019_paper.html

[12] J. Li, A. Louri, A. Karanth, and R. Bunescu, "GCNAX: A flexible and energy-efficient accelerator for graph convolutional neural networks," in *IEEE International Symposium on High-Performance Computer Architecture*, 2021, pp. 775–788. [Online]. Available: https://ieeexplore.ieee.org/document/9407104

[13] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/8999595

[14] Z. Liu, K. Zhou, F. Yang, L. Li, R. Chen, and X. Hu, "EXACT: Scalable graph neural networks training via extreme activation compression," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=vkaMaq95_rX

[15] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible DRAM simulator," *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10321647

[16] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi, "CACTI 6.0: A tool to model large caches," HP Laboratories, Tech. Rep. HPL-2009-85, 2009. [Online]. Available: https://shiftleft.com/mirrors/www.hpl.hp.com/techreports/2009/HPL-2009-85.html

[17] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "SCNN: An accelerator for compressed-sparse convolutional neural networks," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017, pp. 27–40. [Online]. Available: https://dl.acm.org/doi/10.1145/3079856.3080254

[18] G. Pekhimenko, V. Seshadri, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "Base-delta-immediate compression: Practical data compression for on-chip caches," in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques*, 2012, pp. 377–388. [Online]. Available: https://dl.acm.org/doi/10.1145/2370816.2370870

[19] M. Rhu, M. O'Connor, N. Chatterjee, J. Pool, Y. Kwon, and S. W. Keckler, "Compressing DMA engine: Leveraging activation sparsity for training deep neural networks," in *IEEE International Symposium on High Performance Computer Architecture*, 2018, pp. 78–91. [Online]. Available: https://research.nvidia.com/publication/2018-02_compressing-dma-engine-leveraging-activation-sparsity-training-deep-neural

[20] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "SCALE-Sim: Systolic CNN accelerator simulator," *arXiv preprint arXiv:1811.02883*, 2018. [Online]. Available: https://arxiv.org/abs/1811.02883

[21] Z. Shao, X. Chen, L. Du, L. Chen, Y. Du, W. Zhuang, H. Wei, C. Xie, and Z. Wang, "Memory-efficient CNN accelerator based on interlayer feature map compression," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 2, pp. 668–681, 2022. [Online]. Available: https://arxiv.org/abs/2110.06155

[22] Y. Wang, X. Pan, Y. An, J. Zhang, and G. Reinman, "BeaconGNN: Large-scale GNN acceleration with out-of-order streaming in-storage computing," in *IEEE International Symposium on High-Performance Computer Architecture*, 2024, pp. 330–344. [Online]. Available: https://ieeexplore.ieee.org/document/10476427

[23] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "HyGCN: A GCN accelerator with hybrid architecture," in *IEEE International Symposium on High Performance Computer Architecture*, 2020, pp. 15–29. [Online]. Available: https://ieeexplore.ieee.org/document/9065592

[24] M. Yoo, J. Song, J. Lee, N. Kim, Y. Kim, and J. Lee, "SGCN: Exploiting compressed-sparse features in deep graph convolutional network accelerators," in *IEEE International Symposium on High-Performance Computer Architecture*, 2023, pp. 1–14. [Online]. Available: https://ieeexplore.ieee.org/document/10071102

[25] Z. Yue, X. Song, T. Liu, X. Hu, R. Zhang, Z. Du, W. Li, Q. Guo, and T. Chen, "Cambricon-DG: An accelerator for redundancy-free dynamic graph neural networks based on nonlinear isolation," in *IEEE International Symposium on High-Performance Computer Architecture*, 2025, pp. 934–948. [Online]. Available: https://ieeexplore.ieee.org/document/10946746

[26] J. Zhang, P. Raj, S. Zarar, A. Ambardekar, and S. Garg, "CompAct: On-chip compression of activations for low-power systolic array-based CNN acceleration," *ACM Transactions on Embedded Computing Systems*, vol. 18, no. 5s, pp. 47:1–47:24, 2019. [Online]. Available: https://dl.acm.org/doi/10.1145/3358178

APPENDIX: AI USE

OpenAI ChatGPT (GPT-5.6 Thinking) was used in the following parts of the work. For the Abstract, Introduction, Related Work, and Conclusion, it assisted with organization, sentence-level rewriting, and consistency editing. For the Representation, Analysis, Microarchitecture, and Evaluation sections, it assisted with exposition, notation checks, and integration of author-supplied result tables; the authors independently verified every equation, proof statement, configuration, and numerical claim. It assisted with Python plotting code for Figs. 1–8 and the supplementary figures; the authors checked every plotted point against the packaged source CSV and regenerated the figures locally. It assisted with review-response organization, experiment specifications, and debugging suggestions for evaluation scripts; all experiments were run by the authors, and the authors inspected the source code, tests, logs, and manifests before retaining results. No citation or measurement was accepted without author verification against the supplied source material.